

Advance Web Development Lab Record (CSEB4751P)

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Name: Priya Singla

Roll No.: 12201230

Class: 4CE5

Registration No.: 7141-2022-317



Submitted to:

Dr. Neelofar Sohi

Assistant Professor

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PUNJABI UNIVERSITY
PATIALA - 147002**

INDEX

PROJECT INTRODUCTION.....	3
Key Features :-	3
Technical Architecture :-	3
TECHNOLOGY STACK OVERVIEW	4
HTML.....	5
1.Introduction:	5
2.Role of HTML in the Project:	5
3. Structure of the Pages:	5
4.Key HTML Features Used:	5
CSS.....	7
1. Introduction:	7
2. Role of CSS in the Project:	7
3. Styling Structure:	7
4. Key CSS Features Used:	8
JAVASCRIPT	9
1. Introduction:	9
2. Role of JavaScript in the Project:	9
3. Functionality Implemented Using JavaScript:	9
4. Key JavaScript Concepts Used:	9
NODE.JS.....	11
1. Introduction:	11
2. Role of Node.js in the Project:	11
3. Functionality Implemented Using Node.js:	11
4. Key Node.js Concepts Used:	11
MONGODB	12
1. Introduction:	12
2. Role of MongoDB in the Project:	12
3. Functionality Implemented Using MongoDB:	12
4. Key MongoDB Concepts Used:	12
SOURCE CODE	13
SCREENSHOTS	37

PROJECT INTRODUCTION

The **Blog Website** is a full-stack web development project that allows users to create, edit, and share their own blog posts online. The primary objective of this project is to provide a simple, user-friendly, and dynamic platform for individuals to express their thoughts, ideas, and experiences through written content.

Unlike traditional blogging platforms that require extensive setup, this project focuses on ease of use and efficient content management. It integrates both frontend and backend technologies to ensure a seamless experience from the user interface to data storage.

Users can register and log in securely, write blog posts, upload images, view others' posts, and modify or delete their own posts. The application also implements **CRUD (Create, Read, Update, Delete)** operations using Node.js and MongoDB.

The frontend uses **HTML, CSS, and JavaScript** for structure, design, and interactivity, while the backend is powered by **Node.js** with **Express.js** for handling requests and **MongoDB** for database management.

This project demonstrates how multiple web technologies can be integrated to build a functional and aesthetic web application. The user interface includes a **responsive homepage** with gradient design, a **new post form**, and **edit/delete features**, offering an experience similar to real blogging platforms.

Key Features :-

- User Authentication (Register, Login, Logout)
- Create, Edit, Delete, and View Blog Posts
- Image Upload and Display Functionality
- Real-Time Post Rendering and Updates
- Secure Data Storage with MongoDB

Technical Architecture :-

Frontend (HTML, CSS, JavaScript) → Backend (Node.js HTTP Server) → MongoDB Database → Client-Side Rendering of Blog Posts

TECHNOLOGY STACK OVERVIEW

The Blog Website uses a **MERN-like structure** (without React), combining essential frontend and backend technologies for smooth communication and performance.

➤ Frontend:

- **HTML (Hyper Text Markup Language)** – Defines the structure and content of web pages.
- **CSS (Cascading Style Sheets)** – Styles and layouts the pages, including gradients and responsive design.
- **JavaScript** – Adds interactivity, handles user inputs, and connects with the backend API.

➤ Backend:

- **Node.js** – Handles server-side logic, APIs, and routes.
- **Express.js** – Simplifies routing, middleware, and HTTP handling.

➤ Database:

- **MongoDB** – Stores users, posts, and image data efficiently.

➤ Tools and Packages:

- **Nodemon** – For automatic server restarts during development.
- **Multer** – For image upload handling.
- **JWT (JSON Web Token)** – For authentication.
- **Mongoose** – For interacting with MongoDB.

This combination provides a powerful environment for modern web development — flexible, efficient, and scalable.

HTML

1.Introduction:

HTML (Hyper Text Markup Language) is the backbone of the Blog Website's structure. It defines how the content is displayed on the web browser, including headings, paragraphs, images, buttons, and forms. Every page in the project — such as **Home, New Post, Edit Post, Login, and Register** — is created using HTML, providing a semantic and organized layout that browsers can interpret easily.

2.Role of HTML in the Project:

In the Blog Website, HTML serves as the **foundation of every page**, determining the layout of components like:

- Navigation bar and header section
- Post creation and editing forms
- Blog cards displaying title, image, and content preview
- Buttons for actions like “Read”, “Edit”, and “Delete”
- Authentication forms for login and registration

Each of these sections is built with semantic HTML tags (<header>, <main>, <section>, <form>, <button>, etc.) for clarity and accessibility.

3. Structure of the Pages:

The project follows a clean and modular HTML structure for better maintainability:

- **index.html:** Displays all blog posts in a grid layout.
- **newpost.html:** Contains a form for creating new posts.
- **editpost.html:** Enables editing existing posts and uploading new images.
- **login.html / register.html:** Manages user authentication.

Each page links to the same **main.css** file for styling and **app.js** for dynamic content loading.

4.Key HTML Features Used:

- **Forms:** Used for creating and editing blog posts (<form>, <input>, <textarea>, <button>).
- **Media Elements:** is used to display blog images uploaded by users.

- **Navigation Bar:** Built using `<header>` and `<nav>` elements for site navigation.
- **Responsive Layout:** HTML elements are structured with classes to allow flexible design across devices.
- **Dynamic Integration:** JavaScript accesses HTML elements through their IDs (`document.getElementById`) for real-time updates, such as fetching and rendering posts dynamically.

CSS

1. Introduction:

CSS (Cascading Style Sheets) is used to design and enhance the visual presentation of the Blog Website. While HTML defines the structure, CSS adds **styling, layout control, colors, typography, and responsiveness**, creating a user-friendly and modern interface. It ensures that all pages—Home, New Post, Edit Post, Login, and Register—maintain a consistent and attractive design.

2. Role of CSS in the Project:

CSS controls how the entire blog application looks and feels. It is responsible for:

- Setting background gradients, colors, and font styles.
- Designing navigation bars, buttons, and cards.
- Creating a responsive layout for various devices.
- Adding hover effects and box shadows for interactivity.
- Aligning forms, input fields, and hero sections for better usability.

By using a single **main.css** file, styling consistency is maintained throughout the website.

3. Styling Structure:

The stylesheet is organized into multiple sections:

- **Global Styles:** Variables (--primary, --bg, etc.) and base rules for typography, padding, and margins.
- **Topbar Section:** Gradient background for the navbar with flex alignment.
- **Hero Section:** A background image overlaid with a gradient to make the homepage visually appealing.
- **Post Cards:** Designed with box shadows, rounded corners, and hover animations.
- **Forms (New/Edit Posts):** Styled with spacing, rounded inputs, and drop shadows for better readability.

4. Key CSS Features Used:

- **CSS Variables:** For consistent color theming and easy updates.
- **Flexbox & Grid:** To arrange navigation, posts, and form elements responsively.
- **Linear Gradients:** Used for the navbar and hero backgrounds to make them visually appealing.
- **Transitions & Hover Effects:** For smooth button and card animations.
- **Media Queries (optional expansion):** Helped ensure responsiveness across mobile, tablet, and desktop views.

JAVASCRIPT

1. Introduction:

JavaScript (JS) is the **core scripting language** used in the Blog Website to add interactivity, dynamic content loading, and communication between the frontend and backend. While HTML structures and CSS styles the page, JavaScript makes it *interactive and functional* — handling user actions, fetching blog data, and updating the DOM without reloading the page.

2. Role of JavaScript in the Project:

In this Blog Website, JavaScript is used to manage the behavior and logic of the application. Its main roles include:

- Handling **user authentication** (login and register).
- Fetching blog posts from the backend dynamically.
- Managing **Create, Read, Update, and Delete (CRUD)** operations for posts.
- Displaying post details when the “Read” button is clicked.
- Validating form inputs before submission.
- Handling **image rendering** and responsive content updates.

3. Functionality Implemented Using JavaScript:

- **Fetch API Integration:** JavaScript communicates with the backend API to retrieve or send data using `fetch()`.
- **Dynamic DOM Manipulation:** Blog posts are dynamically displayed or removed from the page.
- **Event Listeners:** Detects user interactions like button clicks and form submissions.
- **Form Validation:** Ensures fields like title, content, and image are correctly filled before submission.
- **Navigation Control:** Redirects users to different pages after successful login, logout, or posting.

4. Key JavaScript Concepts Used:

- **Async/Await:** Used to handle asynchronous API calls cleanly.
- **Promises:** Manage data retrieval and error handling efficiently.

- **LocalStorage:** Temporarily stores user login tokens or session details.
- **Modules (Optional):** Code can be organized into separate files for clarity.
- **Error Handling:** Used try...catch blocks to manage backend response errors gracefully.

NODE.JS

1. Introduction:

Node.js is an **open-source, server-side JavaScript runtime** built on Chrome's V8 engine. It allows developers to use JavaScript for backend development, enabling the same language to be used on both client and server sides. In this Blog Website, Node.js powers the backend, handles data flow between the frontend and database, and manages all CRUD operations (Create, Read, Update, Delete) for blog posts.

2. Role of Node.js in the Project:

Node.js acts as the **server-side environment** that executes JavaScript code outside of the browser. It processes client requests, interacts with the database, and sends back responses. Key roles include:

- Serving as the backend runtime for API requests.
- Handling **HTTP requests and responses**.
- Managing routes for user authentication and post operations.
- Interacting with the database (MongoDB) to store and retrieve blog data.
- Managing file uploads for blog images using middleware.

3. Functionality Implemented Using Node.js:

- **Post Management:** Add, edit, delete, and fetch blog posts.
- **Authentication:** Handles secure user registration and login using JWT (JSON Web Tokens).
- **File Handling:** Manages blog images uploaded by users.
- **Middleware Integration:** Uses middleware for tasks like authentication and file uploads.
- **API Endpoints:** Provides RESTful APIs consumed by the frontend via fetch() calls.

4. Key Node.js Concepts Used:

- **Modules:** Reusable components using require() and module.exports.
- **Middleware:** Executes specific functions during the request-response cycle.
- **Routing:** Organizes API endpoints logically (e.g., /auth, /posts).

MONGODB

1. Introduction:

MongoDB is a **NoSQL document-oriented database** that stores data in flexible, JSON-like documents instead of traditional tables. It is designed for scalability, performance, and high availability. In this Blog Website project, MongoDB is used to store all blog-related data, including user accounts, posts, images, and authentication tokens, ensuring seamless and efficient data management.

2. Role of MongoDB in the Project:

MongoDB serves as the **primary database** for the blog application. It enables efficient data storage and retrieval without requiring a fixed schema, making it ideal for dynamic web applications.

Its roles include:

- Storing **user information** (name, email, password).
- Maintaining **blog post records** (title, content, image path, creation date, author).
- Handling relationships between users and their posts.
- Supporting **CRUD operations** through Mongoose models.

3. Functionality Implemented Using MongoDB:

- **Create:** Saves new blog posts, users, and uploaded image paths.
- **Read:** Fetches posts for display on the homepage or detailed view.
- **Update:** Allows editing blog titles, content, or images.
- **Delete:** Enables removing posts or users securely.
- **Authentication Support:** Stores encrypted passwords and tokens for user login.

4. Key MongoDB Concepts Used:

- **Collections & Documents:** Data is stored as documents within collections (e.g., Users, Posts).
- **CRUD Operations:** Used throughout the backend for managing blog data.
- **Indexes:** Enable faster searches and filtering when fetching posts or users.

SOURCE CODE

1.HTML

- **Index.html**

```
<!doctype html>
<html>
<head>
<meta charset="utf-8" />
<title>Blog Home</title>
<link rel="stylesheet" href="main.css" />
</head>
<body>
<header class="topbar">
<h1>My Blog</h1>
<nav id="nav">
<button id="btn-new" class="btn">New Post</button>
<span id="user-area"></span>
</nav>
</header>
<!-- Hero Section changed -->
<section class="hero">
<div class="hero-content">
<h2>Welcome to My Blog</h2>
<p>Discover stories, ideas, and insights shared by our creative minds.
Explore topics that inspire, educate, and entertain you every day.</p>
<button id="explore-btn" class="btn hero-btn">Explore Posts</button>
</div>
</section>
<main class="container">
<section id="posts"></section>
</main>
<script src="app.js"></script>
<script>
document.getElementById('btn-new').addEventListener('click', () => {
const token = localStorage.getItem('token');
if (!token) return location.href = 'login.html';
location.href = 'newpost.html';
});
// changed
document.getElementById('explore-btn').addEventListener('click', () => {
document.querySelector('.container').scrollIntoView({ behavior: 'smooth' });
});
</script>
</body>
</html>
```

- **Login.html**

```
<!doctype html>
<html>
<head>
```

```

<meta charset="utf-8" />
<title>Login - Blog</title>
<link rel="stylesheet" href="main.css" />
</head>
<body>
<main class="auth-container">
<h2>Login</h2>
<form id="loginForm">
<input type="email" id="email" placeholder="Email" required />
<input type="password" id="password" placeholder="Password" required />
<button class="btn" type="submit">Login</button>
</form>
<p>Don't have an account? <a href="register.html">Register</a></p>
<p id="msg" class="msg"></p>
</main>
<script>
const API_BASE = "http://localhost:4000/api";
document.getElementById("loginForm").addEventListener("submit", async
(e) => {
e.preventDefault();
const email = document.getElementById("email").value.trim();
const password = document.getElementById("password").value.trim();
const msg = document.getElementById("msg");
msg.textContent = "Logging in...";
try {
const res = await fetch(`${API_BASE}/auth/login`, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ email, password }),
});
const data = await res.json();
if (!res.ok) throw new Error(data.message || "Login failed");
localStorage.setItem("token", data.token);
msg.style.color = "green";
msg.textContent = "Login successful! Redirecting...";
setTimeout(() => location.href = "index.html", 1000);
} catch (err) {
msg.style.color = "red";
msg.textContent = err.message;
}
});
</script>
</body>
</html>

```

- **Editpost.html**

```

<!doctype html>
<html>
<head>

```

```

<meta charset="utf-8" />
<title>Edit Post - Blog</title>
<link rel="stylesheet" href="main.css" />
</head>
<body>
<!-- Prevents navbar from being rendered again by app.js -->
<script>window.skipRenderNav = true;</script>
<!-- Keep only one header/nav -->
<header class="topbar">
<h1>Edit Post</h1>
</header>
<main class="container">
<form id="editForm" class="post-form">
<input type="text" id="title" placeholder="Title" required />
<textarea id="content" placeholder="Content" rows="10"
required></textarea>
<input type="file" id="image" accept="image/*" />
<img id="preview" class="post-img" style="display:none;" />
<button class="btn" type="submit">Save Changes</button>
<button type="button" id="btn-delete" class="btn danger">Delete</button>
<p id="msg" class="msg"></p>
</form>
</main>
<script src="app.js"></script>
<script>
const params = new URLSearchParams(location.search);
const postId = params.get('id');
(async () => {
const token = localStorage.getItem('token');
if (!postId) return location.href = 'index.html';
const post = await fetchPost(postId);
if (!post) return location.href = 'index.html';
document.getElementById('title').value = post.title;
document.getElementById('content').value = post.content;
// Show current image
if (post.image) {
const imgPreview = document.getElementById('preview');
imgPreview.src = `http://localhost:4000${post.image}`;
imgPreview.style.display = 'block';
}
})();
document.getElementById('editForm').addEventListener('submit', async (e)
=> {
e.preventDefault();
const title = document.getElementById('title').value.trim();
const content = document.getElementById('content').value.trim();
const file = document.getElementById('image').files[0];
let imagePath = null;
if (file) {
const form = new FormData();

```

```

form.append('image', file);
const token = localStorage.getItem('token');
const res = await fetch(API_BASE + '/upload', { method: 'POST', headers: {
'Authorization': 'Bearer ' + token }, body: form });
const j = await res.json();
imagePath = j.path;
}
const res2 = await updatePost(postId, { title, content, image: imagePath });
const msg = document.getElementById('msg');
if (res2 && res2._id) location.href = 'index.html';
else msg.textContent = res2?.message || 'Could not update';
});
// Fixed delete button logic
document.getElementById('btn-delete').addEventListener('click', async () => {
if (!confirm('Delete this post?')) return;
const token = localStorage.getItem('token');
const res = await fetch(API_BASE + '/posts/' + postId, {
method: 'DELETE',
headers: { 'Authorization': 'Bearer ' + token }
});
const data = await res.json();
if (res.ok) {
alert('Post deleted successfully');
location.href = 'index.html';
} else {
alert('Could not delete');
}
});
</script>
</body>
</html>

```

- **Newpost.html**

```

<!doctype html>
<html>
<head>
<meta charset="utf-8" />
<title>New Post - Blog</title>
<link rel="stylesheet" href="main.css" />
</head>
<body>
<header class="topbar">
<h1>New Post</h1>
<nav>
<button onclick="logout()" class="btn">Logout</button>
</nav>
</header>
<main>
<div class="auth-container">

```



```

<h2>Create New Post</h2>
<form id="postForm" enctype="multipart/form-data">
  <input type="text" id="title" placeholder="Title" required />
  <textarea id="content" placeholder="Content" rows="10"
required></textarea>
  <input type="file" id="image" accept="image/*" />
  <button class="btn" type="submit">Publish</button>
  <p id="msg" class="msg"></p>
</form>
</div>
</main>
<script src="app.js"></script>
<script>
(async () => {
  const token = localStorage.getItem('token');
  if (!token) location.href = 'login.html';
})();
document.getElementById('postForm').addEventListener('submit', async (e)
=> {
  e.preventDefault();
  const title = document.getElementById('title').value.trim();
  const content = document.getElementById('content').value.trim();
  const file = document.getElementById('image').files[0];
  let imagePath = null;
  if (file) {
    const form = new FormData();
    form.append('image', file);
    const token = localStorage.getItem('token');
    const res = await fetch(API_BASE + '/upload', { method: 'POST', headers: {
'Authorization': 'Bearer ' + token }, body: form });
    const j = await res.json();
    imagePath = j.path;
  }
  const res2 = await createPost({ title, content, image: imagePath });
  const msg = document.getElementById('msg');
  if (res2 && res2._id) location.href = 'index.html';
  else msg.textContent = res2?.message || 'Could not create post';
});
function logout() { localStorage.removeItem('token'); location.href =
'index.html'; }
</script>
</body>
</html>

```

- **Post.html**

```

<!doctype html>
<html>
<head>
<meta charset="utf-8" />
<title>View Post</title>

```

```

<link rel="stylesheet" href="main.css" />
</head>
<body>
<header class="topbar"><h1>Post</h1></header>
<main class="container">
<article id="postArea"></article>
</main>
<script src="app.js"></script>
<script>
const params = new URLSearchParams(location.search);
const postId = params.get('id');
(async () => {
  if (!postId) return location.href = 'index.html';
  const post = await fetchPost(postId);
  if (!post) return location.href = 'index.html';
  const area = document.getElementById('postArea');
  area.innerHTML = `
<h2>${escapeHtml(post.title)}</h2>
<div class="meta">by ${escapeHtml(post.user?.name || 'Unknown')} • ${new
Date(post.createdAt).toLocaleString()}</div>
${post.image ? `` : ""};
<div class="content">${escapeHtml(post.content)}</div>
<div class="actions">
<button class="btn" id="likeBtn">Like (${post.likes.length})</button>
${parseUserIdFromToken() === (post.user && post.user._id) ? `<a
class="btn" href="editpost.html?id=${post._id}">Edit</a>` : ""}
</div>
<section id="comments">
<h3>Comments</h3>
<div id="commentsList"></div>
<form id="commentForm">
<input id="commentText" placeholder="Write a comment" required />
<button class="btn" type="submit">Comment</button>
</form>
</section>
`;
  document.getElementById('likeBtn').addEventListener('click', async () => {
    const token = localStorage.getItem('token');
    if (!token) return location.href = 'login.html';
    const res = await fetch(API_BASE + '/posts/like/' + postId, { method: 'POST',
headers: { 'Authorization': 'Bearer ' + token } });
    const j = await res.json();
    document.getElementById('likeBtn').textContent = 'Like (' + j.likes + ')';
  });
  // render comments
  const cl = document.getElementById('commentsList');
  cl.innerHTML = post.comments.map(c => `<div
class="comment"><strong>${escapeHtml(c.user.name)}</strong>:
${escapeHtml(c.content)}</div>`).join("");

```

```

document.getElementById('commentForm').addEventListener('submit', async
(e) => {
  e.preventDefault();
  const token = localStorage.getItem('token');
  if (!token) return location.href = 'login.html';
  const content = document.getElementById('commentText').value.trim();
  const res = await fetch(API_BASE + '/posts/comment/' + postId, { method:
'POST', headers: { 'Authorization': 'Bearer ' + token, 'Content-Type':
'application/json' }, body: JSON.stringify({ content }) });
  const j = await res.json();
  if (j && j._id) {
    cl.innerHTML += `<div
class="comment"><strong>${escapeHtml(j.user.name)}</strong>:
${escapeHtml(j.content)}</div>`;
    document.getElementById('commentText').value = "";
  }
});
})();
</script>
</body>
</html>

```

- **Register.html**

```

<!doctype html>
<html>
<head>
<meta charset="utf-8" />
<title>Register - Blog</title>
<link rel="stylesheet" href="main.css" />
</head>
<body>
<main class="auth-container">
<h2>Register</h2>
<form id="registerForm">
<input type="text" id="name" placeholder="Your name" required />
<input type="email" id="email" placeholder="Email" required />
<input type="password" id="password" placeholder="Password" required />
<button class="btn" type="submit">Register</button>
</form>
<p>Already have an account? <a href="login.html">Login</a></p>
<p id="msg" class="msg"></p>
</main>
<script src="app.js"></script>
<script>
const form = document.getElementById('registerForm');
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const name = document.getElementById('name').value.trim();
  const email = document.getElementById('email').value.trim();

```

```

const password = document.getElementById('password').value.trim();
const res = await register(name, email, password);
const msg = document.getElementById('msg');
if (res && res.token) {
  localStorage.setItem('token', res.token);
  location.href = 'index.html';
} else {
  msg.textContent = res?.message || 'Registration failed';
}
});
</script>
</body>
</html>

```

2.CSS

```

/* :root{--primary:#2b6cb0;--bg:#f7fafc;--card:#fff;--muted:#6b7280}
*{bo:root {
--primary: #2b6cb0;
--primary-dark: #1a497e;
--bg: #f7fafc;
--card: #fff;
--muted: #6b7280;
}
* {
box-sizing: border-box;
margin: 0;
padding: 0;
}
body {
font-family: "Segoe UI", Roboto, Arial;
background: var(--bg);
color: #111;
overflow-x: hidden;
}
/* Top Navbar */
.topbar {
display: flex;
justify-content: space-between;
align-items: center;
padding: 14px 20px;
background: linear-gradient(90deg, #1e3a8a, #3b82f6);
color: white;
box-shadow: 0 2px 10px rgba(0,0,0,0.15);
}
.topbar h1 {
margin: 0;
font-size: 24px;
letter-spacing: 1px;

```

```

font-weight: 700;
}
.editbar {
background: #2b6cb0; /* same as your navbar color */
color: white;
display: flex;
justify-content: space-between;
align-items: center;
padding: 12px 20px;
box-shadow: 0 1px 2px rgba(0,0,0,0.1);
}
.btn {
background: var(--primary);
color: white;
border: none;
padding: 8px 12px;
border-radius: 6px;
cursor: pointer;
font-weight: 600;
transition: 0.3s;
}
.btn:hover {
background: var(--primary-dark);
}
.btn.danger {
background: #e53e3e;
}
/* Hero Section */
.hero {
background:
linear-gradient(rgba(0,0,0,0.5), rgba(0,0,0,0.5)),
url('https://images.unsplash.com/photo-1507842217343-583bb7270b66?auto=format&fit=crop&w=1600&q=80')
no-repeat center center / cover;
height: 60vh;
display: flex;
justify-content: center;
align-items: center;
text-align: center;
color: white;
padding: 20px;
}
.hero-content {
max-width: 700px;
}
.hero-content h2 {
font-size: 38px;
margin-bottom: 16px;
}
.hero-content p {

```

```

font-size: 18px;
margin-bottom: 20px;
color: #f3f4f6;
}
.hero-btn {
background: #f97316;
}
.hero-btn:hover {
background: #ea580c;
}
/* Post Section */
.container {
max-width: 900px;
margin: 40px auto;
padding: 0 16px;
}
.section-title {
text-align: center;
margin-bottom: 20px;
font-size: 24px;
color: var(--primary-dark);
}
#posts {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 20px;
}
.post-card {
background: var(--card);
padding: 16px;
border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.05);
transition: transform 0.2s, box-shadow 0.2s;
}
.post-card:hover {
transform: translateY(-4px);
box-shadow: 0 4px 16px rgba(0,0,0,0.1);
}
.post-card h3 {
margin: 0 0 6px 0;
}
.post-form {
background: rgba(255, 255, 255, 0.9);
padding: 24px;
border-radius: 10px;
box-shadow: 0 6px 18px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
gap: 10px;
max-width: 600px;

```

```

margin: 50px auto;
}
/* Fix extra navbar issue on edit page */
.editbar {
background: #2b6cb0; /* same as your navbar color */
color: white;
display: flex;
justify-content: space-between;
align-items: center;
padding: 12px 20px;
box-shadow: 0 1px 2px rgba(0,0,0,0.1);
}
.meta {
color: var(--muted);
font-size: 13px;
margin-bottom: 8px;
}
.post-img {
max-width: 100%;
border-radius: 8px;
margin: 12px 0;
object-fit: cover;
height: 180px;
}
.actions {
margin-top: 12px;
display: flex;
gap: 8px;
}
.comment {
padding: 8px 0;
border-bottom: 1px solid #eee;
}
.msg {
color: #922;
margin-top: 8px;
}
/* =====
Fix for New Post / Auth Pages
===== */
.auth-container {
max-width: 420px;
margin: 100px auto;
background: white;
padding: 28px;
border-radius: 10px;
box-shadow: 0 6px 20px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
}

```

```

.auth-container h2 {
text-align: center;
margin-bottom: 20px;
color: var(--primary-dark);
}
.auth-container label {
font-weight: 600;
margin-top: 10px;
display: block;
}
.auth-container input,
.auth-container textarea,
.auth-container select {
width: 100%;
padding: 10px 12px;
margin: 8px 0 14px;
border: 1px solid #cbd5e1;
border-radius: 6px;
font-size: 15px;
}
.auth-container textarea {
resize: vertical;
min-height: 100px;
}
.auth-container .btn {
width: 100%;
padding: 10px;
border-radius: 8px;
font-size: 16px;
margin-top: 8px;
}
.auth-container .btn:hover {
transform: translateY(-1px);
}
.auth-container .msg {
text-align: center;
margin-top: 10px;
color: #e53e3e;
font-weight: 500;
}
}
x-sizing:border-box}body{font-family:Segoe
UI,Roboto,Arial;margin:0;background:var(--bg);color:#111}
.topbar{display:flex;justify-content:space-between;align-items:center;padding:12px
20px;background:white;box-shadow:0 1px 2px rgba(0,0,0,0.04)}
.topbar h1{margin:0} .container{max-width:900px;margin:24px auto;padding:0
16px}
.btn{background:var(--primary);color:white;border:none;padding:8px 12px;border-
radius:6px;cursor:pointer;font-weight:600}

```



```

.btn.danger{background:#e53e3e}.auth-container{max-width:380px;margin:80px
auto;background:white;padding:24px;border-radius:8px;box-shadow:0 6px 18px
rgba(0,0,0,0.06)}
input,textarea{width:100%;padding:10px;margin:8px 0;border:1px solid
#e2e8f0;border-radius:6px}
#posts{display:grid;grid-template-columns:1fr;gap:16px}.post-card{background:var(-
-card);padding:16px;border-radius:8px;box-shadow:0 1px 6px rgba(0,0,0,0.04)}
.post-card h3{margin:0 0 6px 0}.meta{color:var(--muted);font-size:13px;margin-
bottom:8px}.actions{margin-top:12px;display:flex;gap:8px}
.post-img{max-width:100%;border-radius:8px;margin:12px
0}.comment{padding:8px 0;border-bottom:1px solid #eee}
.msg{color:#922;margin-top:8px} */
:root {
--primary: #2b6cb0;
--primary-dark: #1a497e;
--bg: #f7fafc;
--card: #fff;
--muted: #6b7280;
}
* {
box-sizing: border-box;
margin: 0;
padding: 0;
}
body {
font-family: "Segoe UI", Roboto, Arial;
background: var(--bg);
color: #111;
overflow-x: hidden;
}
/* Top Navbar */
.topbar {
display: flex;
justify-content: space-between;
align-items: center;
padding: 14px 20px;
background: linear-gradient(90deg, #1e3a8a, #3b82f6);
color: white;
box-shadow: 0 2px 10px rgba(0,0,0,0.15);
}
.topbar h1 {
margin: 0;
font-size: 24px;
letter-spacing: 1px;
font-weight: 700;
}
.editbar {
background: #2b6cb0; /* same as your navbar color */
color: white;
display: flex;

```

```

justify-content: space-between;
align-items: center;
padding: 12px 20px;
box-shadow: 0 1px 2px rgba(0,0,0,0.1);
}
.btn {
background: var(--primary);
color: white;
border: none;
padding: 8px 12px;
border-radius: 6px;
cursor: pointer;
font-weight: 600;
transition: 0.3s;
}
.btn:hover {
background: var(--primary-dark);
}
.btn.danger {
background: #e53e3e;
}
/* Hero Section */
.hero {
background:
linear-gradient(rgba(0,0,0,0.5), rgba(0,0,0,0.5)),
url('https://images.unsplash.com/photo-1507842217343-583bb7270b66?auto=format&fit=crop&w=1600&q=80')
no-repeat center center / cover;
height: 60vh;
display: flex;
justify-content: center;
align-items: center;
text-align: center;
color: white;
padding: 20px;
}
.hero-content {
max-width: 700px;
}
.hero-content h2 {
font-size: 38px;
margin-bottom: 16px;
}
.hero-content p {
font-size: 18px;
margin-bottom: 20px;
color: #f3f4f6;
}
.hero-btn {
background: #f97316;

```

```

}
.hero-btn:hover {
background: #ea580c;
}
/* Post Section */
.container {
max-width: 900px;
margin: 40px auto;
padding: 0 16px;
}
.section-title {
text-align: center;
margin-bottom: 20px;
font-size: 24px;
color: var(--primary-dark);
}
#posts {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 20px;
}
.post-card {
background: var(--card);
padding: 16px;
border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.05);
transition: transform 0.2s, box-shadow 0.2s;
}
.post-card:hover {
transform: translateY(-4px);
box-shadow: 0 4px 16px rgba(0,0,0,0.1);
}
.post-card h3 {
margin: 0 0 6px 0;
}
.post-form {
background: rgba(255, 255, 255, 0.9);
padding: 24px;
border-radius: 10px;
box-shadow: 0 6px 18px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
gap: 10px;
max-width: 600px;
margin: 50px auto;
}
/* Fix extra navbar issue on edit page */
.editbar {
background: #2b6cb0; /* same as your navbar color */
color: white;

```

```

display: flex;
justify-content: space-between;
align-items: center;
padding: 12px 20px;
box-shadow: 0 1px 2px rgba(0,0,0,0.1);
}
.meta {
color: var(--muted);
font-size: 13px;
margin-bottom: 8px;
}
.post-img {
max-width: 100%;
border-radius: 8px;
margin: 12px 0;
object-fit: cover;
height: 180px;
}
.actions {
margin-top: 12px;
display: flex;
gap: 8px;
}
.comment {
padding: 8px 0;
border-bottom: 1px solid #eee;
}
.msg {
color: #922;
margin-top: 8px;
}
/* =====
Fix for New Post / Auth Pages
===== */
.auth-container {
max-width: 420px;
margin: 100px auto;
background: white;
padding: 28px;
border-radius: 10px;
box-shadow: 0 6px 20px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
}
.auth-container h2 {
text-align: center;
margin-bottom: 20px;
color: var(--primary-dark);
}
.auth-container label {

```

```

font-weight: 600;
margin-top: 10px;
display: block;
}
.auth-container input,
.auth-container textarea,
.auth-container select {
width: 100%;
padding: 10px 12px;
margin: 8px 0 14px;
border: 1px solid #cbd5e1;
border-radius: 6px;
font-size: 15px;
}
.auth-container textarea {
resize: vertical;
min-height: 100px;
}
.auth-container .btn {
width: 100%;
padding: 10px;
border-radius: 8px;
font-size: 16px;
margin-top: 8px;
}
.auth-container .btn:hover {
transform: translateY(-1px);
}
.auth-container .msg {
text-align: center;
margin-top: 10px;
color: #e53e3e;
font-weight: 500;
}

```

3.JS

- **app.js**

```

const API_BASE = 'http://localhost:4000/api';
async function register(name,email,password){try{const r=await
fetch(API_BASE+'/auth/register',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({name,email,password})});return await
r.json()}catch(e){return{message:'Network error'}}}
async function login(email,password){try{const r=await
fetch(API_BASE+'/auth/login',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({email,password})});return await
r.json()}catch(e){return{message:'Network error'}}}
function getAuthHeaders(){const token=localStorage.getItem('token');return
token?{'Authorization':'Bearer '+token,'Content-Type':'application/json'}:{'Content-
Type':'application/json'}}

```

```

async function fetchPosts(){try{const r=await fetch(API_BASE+'/posts');return await
r.json()}catch(e){return []}}
async function createPost(data){try{const r=await
fetch(API_BASE+'/posts',{method:'POST',headers:getAuthHeaders(),body:JSON.s
tringify(data)});return await r.json()}catch(e){return {message:'Network error'}}}
async function fetchPost(id){try{const r=await fetch(API_BASE+'/posts/'+id);return
await r.json()}catch(e){return null}}
async function updatePost(id,data){try{const r=await
fetch(API_BASE+'/posts/'+id,{method:'PUT',headers:getAuthHeaders(),body:JSON.s
tringify(data)});return await r.json()}catch(e){return {message:'Network error'}}}
async function deletePost(id){try{const r=await
fetch(API_BASE+'/posts/'+id,{method:'DELETE',headers:getAuthHeaders()});return
await r.json()}catch(e){return {message:'Network error'}}}
function escapeHtml(t){if(!t) return "";return
t.replace(/[\&<>"]/g,m=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;','"':'&#39;
'})[m])}
function truncate(s,n){if(!s) return "";return s.length>n? s.slice(0,n)+'...': s}
function parseUserIdFromToken(){try{const
token=localStorage.getItem('token');if(!token) return null;return
JSON.parse(atob(token.split('.')[1])).user.id}catch(e){return null}}
async function renderNav(){const ua=document.getElementById('user-area'); if(!ua)
return; const token=localStorage.getItem('token'); if(token){ua.innerHTML=`<button
class="btn"
id="logoutBtn">Logout</button>`;document.getElementById('logoutBtn').addEventL
istener('click',()=>{localStorage.removeItem('token');location.reload()})}else{ua.inne
rHTML=`<a href="login.html" class="btn">Login</a>`}
async function renderPostsOnHome(){const
postsEl=document.getElementById('posts'); if(!postsEl) return; const posts=await
fetchPosts(); if(!Array.isArray(posts)){postsEl.innerHTML=`<p>Could not load
posts.</p>`;return} if(posts.length===0){postsEl.innerHTML=`<p>No posts yet. <a
href="newpost.html">Create the first post</a></p>`;return} postsEl.innerHTML =
posts.map(p=>{const date=new Date(p.createdAt).toLocaleString(); const
author=p.user && p.user.name; return `
<article class="post-card">
<h3>${escapeHtml(p.title)}</h3>
<div class="meta">by ${escapeHtml(author||'Unknown')} • ${date}</div>
${p.image ? `` : ""}
<div class="excerpt">${escapeHtml(truncate(p.content,300))}</div>
<div class="actions">
<a class="btn" href="post.html?id=${p._id}>Read</a>
${parseUserIdFromToken()=== (p.user && p.user._id) ? `<a class="btn"
href="editpost.html?id=${p._id}>Edit</a>` : ""}
</div>
</article>`}).join("");}
document.addEventListener('DOMContentLoaded', async ()=>{
if (!window.skipRenderNav) {
await renderNav();
await renderPostsOnHome();
}
});

```

- **Server.js**

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const path = require("path");
const fs = require("fs");
require("dotenv").config();
const app = express(); // <-- create app first
// ensure uploads folder exists
const uploadsDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadsDir)) fs.mkdirSync(uploadsDir);
// middlewares
app.use(cors({
  origin: "http://127.0.0.1:5500", // or your frontend port
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));
app.use(express.json());
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
// routes
app.use("/api/auth", require("./routes/auth"));
app.use("/api/posts", require("./routes/posts"));
app.use("/api/upload", require("./routes/upload"));
// connect to MongoDB
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("MongoDB connected"))
  .catch(err => console.log("MongoDB connection error:", err));
// start server
const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

4.Routes

- **Upload.js**

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const auth = require('../middleware/auth');
const router = express.Router();
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, path.join(__dirname, '../uploads'));
  },
  filename: function (req, file, cb) {
    const ext = path.extname(file.originalname);
    cb(null, Date.now() + ext);
  }
});
const upload = multer({ storage });
router.post('/', auth, upload.single('image'), (req, res) => {
```

```

if (!req.file) return res.status(400).json({ message: 'No file uploaded' });
res.json({ path: `/uploads/${req.file.filename}` });
});
module.exports = router;

```

- **Posts.js**

```

const express = require('express');
const auth = require('../middleware/auth');
const Post = require('../models/Post');
const Comment = require('../models/Comment');
const router = express.Router();

// create post
router.post('/', auth, async (req, res) => {
  try {
    const { title, content, image } = req.body;
    if (!title || !content) return res.status(400).json({ message: 'Title and content required' });
    const post = new Post({ user: req.user.id, title, content, image });
    await post.save();
    res.json(post);
  } catch (err) {
    console.error(err); res.status(500).json({ message: 'Server error' });
  }
});

// get all posts
router.get('/', async (req, res) => {
  try {
    const posts = await Post.find().populate('user', 'name').populate({ path: 'comments',
    populate: { path: 'user', select: 'name' } }).sort({ createdAt: -1 });
    res.json(posts);
  } catch (err) { console.error(err); res.status(500).json({ message: 'Server error' }); }
});

// get single post
router.get('/:id', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id).populate('user', 'name').populate({
    path: 'comments', populate: { path: 'user', select: 'name' } });
    if (!post) return res.status(404).json({ message: 'Post not found' });
    res.json(post);
  } catch (err) { console.error(err); res.status(500).json({ message: 'Server error' }); }
});

// update post
router.put('/:id', auth, async (req, res) => {
  try {
    const { title, content, image } = req.body;
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    if (post.user.toString() !== req.user.id) return res.status(403).json({ message: 'Not
    authorized' });
    post.title = title || post.title;

```



```

post.content = content || post.content;
post.image = image || post.image;
await post.save();
res.json(post);
} catch (err) { console.error(err); res.status(500).json({ message: 'Server error' }); }
});
// delete post
router.delete('/:id', auth, async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    await Post.findByIdAndDelete(req.params.id);
    res.json({ message: 'Post deleted successfully' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
// like / unlike post
router.post('/like/:id', auth, async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    const idx = post.likes.findIndex(id => id.toString() === req.user.id);
    if (idx === -1) {
      post.likes.push(req.user.id);
    } else {
      post.likes.splice(idx, 1);
    }
    await post.save();
    res.json({ likes: post.likes.length });
  } catch (err) { console.error(err); res.status(500).json({ message: 'Server error' }); }
});
// add comment
router.post('/comment/:id', auth, async (req, res) => {
  try {
    const { content } = req.body;
    if (!content) return res.status(400).json({ message: 'Content required' });
    const post = await Post.findById(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    const comment = new Comment({ user: req.user.id, post: post._id, content });
    await comment.save();
    post.comments.push(comment._id);
    await post.save();
    const populated = await comment.populate('user', 'name');
    res.json(populated);
  } catch (err) { console.error(err); res.status(500).json({ message: 'Server error' }); }
});
module.exports = router;

```

- **Auth.js**

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const router = express.Router();
// Test route (check if backend works)
router.get('/test', (req, res) => {
  res.send(" Auth route working!");
});
// Register route
router.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;
    if (!name || !email || !password)
      return res.status(400).json({ message: 'Please fill all fields' });
    let user = await User.findOne({ email });
    if (user)
      return res.status(400).json({ message: 'User already exists' });
    const salt = await bcrypt.genSalt(10);
    const hashed = await bcrypt.hash(password, salt);
    user = new User({ name, email, password: hashed });
    await user.save();
    const payload = { user: { id: user.id } };
    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'Server error' });
  }
});
// Login route
router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password)
      return res.status(400).json({ message: 'Please fill all fields' });
    const user = await User.findOne({ email });
    if (!user)
      return res.status(400).json({ message: 'Invalid credentials' });
    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch)
      return res.status(400).json({ message: 'Invalid credentials' });
    const payload = { user: { id: user.id } };
    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '7d' });
    res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
  } catch (err) {
    console.error(err);
    res.status(500).json({ message: 'Server error' });
  }
});
```

```
});  
module.exports = router;
```

5.Models

- **Comment.js**

```
const mongoose = require('mongoose');  
const CommentSchema = new mongoose.Schema({  
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },  
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post', required: true },  
  content: { type: String, required: true }  
}, { timestamps: true });  
module.exports = mongoose.model('Comment', CommentSchema);
```

- **User.js**

```
const mongoose = require('mongoose');  
const UserSchema = new mongoose.Schema({  
  name: { type: String, required: true, trim: true },  
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },  
  password: { type: String, required: true }  
}, { timestamps: true });  
module.exports = mongoose.model('User', UserSchema);
```

- **Post.js**

```
const mongoose = require('mongoose');  
const PostSchema = new mongoose.Schema({  
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },  
  title: { type: String, required: true, trim: true },  
  content: { type: String, required: true },  
  image: { type: String }, // path to uploaded image  
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],  
  comments: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Comment' }]  
}, { timestamps: true });  
module.exports = mongoose.model('Post', PostSchema);
```

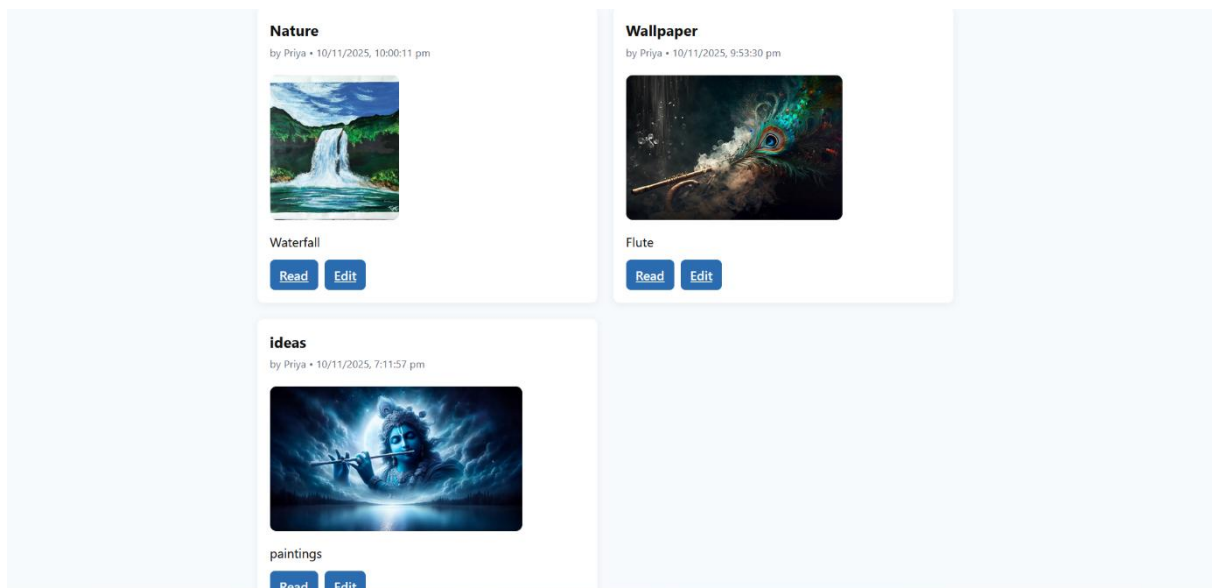
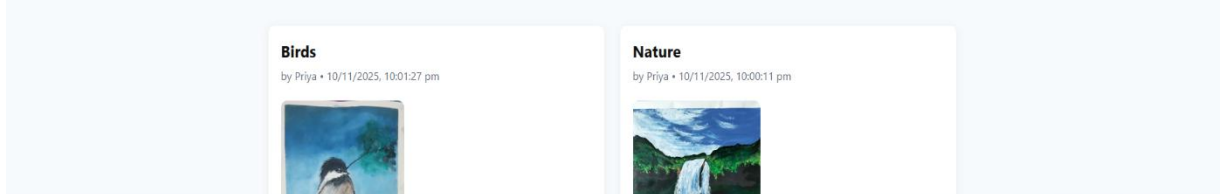
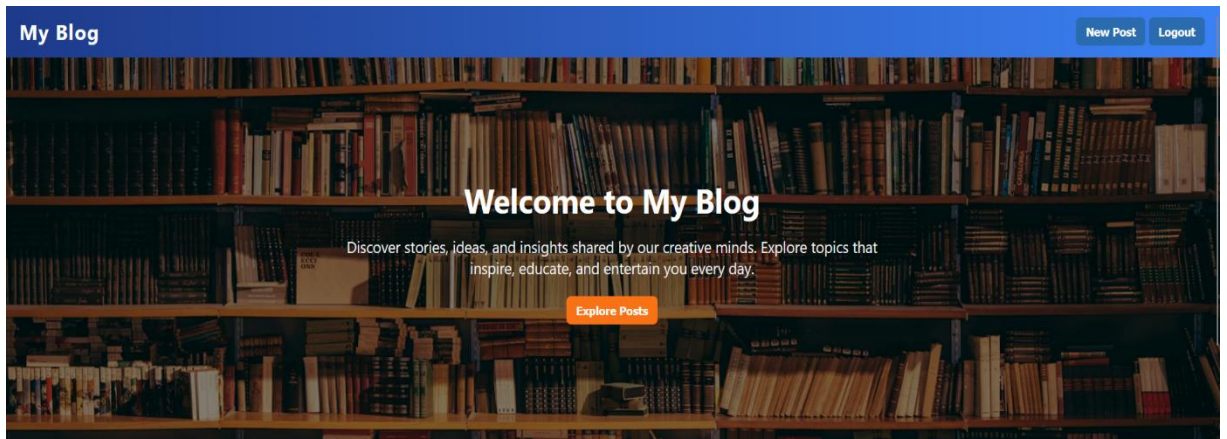
6.Middleware

- **Auth.js**

```
const jwt = require('jsonwebtoken');  
module.exports = (req, res, next) => {  
  const authHeader = req.header('Authorization');  
  if (!authHeader) return res.status(401).json({ message: 'No token, authorization denied' });  
  const token = authHeader.split(' ')[1];  
  if (!token) return res.status(401).json({ message: 'No token, authorization denied' });  
  try {  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = decoded.user;  
    next();  
  }  
}
```

```
    } catch (err) {  
      res.status(401).json({ message: 'Token is not valid' });  
    }  
  };  
}
```

SCREENSHOTS



Login

Email

Password

Login

Don't have an account? [Register](#)

New PostLogout

Create New Post

Title

Content

Browse...

No file selected.

Publish

Edit Post

Nature

WaterFall

Browse...

No file selected.



Save Changes

Delete

Post

Nature

by Priya • 10/11/2025, 10:00:11 pm



Waterfall

Like (0) Edit

Comments

Write a comment Comment